

Hyper Text Transfer Protocol (HTTP): Network protocol exchanging data between clients and server. It follows a technical specification called *Request From Comment (RFC)*, There are a ton of RFC specifications identified by numbers. The common point among them is that they define the specifications of the Internet and only the Internet. HTTP is defined by **RFC 7231**. RFC 7231 can be found at this address: [1https://www.rfc-editor.org/rfc/rfc7231.html](https://www.rfc-editor.org/rfc/rfc7231.html).

Data Format

1. Extensible Markup Language (XML)
2. JavaScript Object Notation (JSON)
3. Multipurpose Mail Extensions (MIME)

The Characteristics of HTTP

HTTP has three essential characteristics:

1. It is **stateless**: This means that after sending a request to the server and receiving the response, neither the client nor the server retains any information on the exchanged request.
2. It is **connectionless**: An HTTP connection is open between the client and the server. Once the client has received the response from the server, the connection is closed, and the connection between the two systems is not permanent.
3. It is **independent** of the media, that is, the server can transmit any media as long as the client and the server “agree” on exchanging the content.

Cookies: Most web applications need to recognize the same user during a browsing session (identify this user through browsing between several web pages). To achieve this, an RFC describes HTTP cookies designed to keep user data browser-side.

HTTP Requests and Responses

An HTTP connection works as follows: a request will receive a response unless the connection is broken. Every request and every response works the same way, and I’ll go into more detail in the next section.

An HTTP request works with elements as follows:

A client (a browser, an application) initiates an HTTP request by invoking a **URI**, the address of the requested resource on the server.

The URI requires the use of a **verb** that will determine the action to be performed.

Metadata will be sent in the HTTP request, called **headers**. These headers allow controlling the content negotiated with the server, such as sending authentication information and much more.

Parameters are necessary to exchange content with the server and obtain the response sought, and the parameters can be in the request's body, the route, or the URI.

An HTTP response works with elements as follows:

- The server returns a response with a simple status code (HTTP status code) to determine how the HTTP request processing took place.
- The server also returns headers in response to the client providing with different metadata.
- Finally, the server will return (or not) a payload formatted in the MIME type requested by the client.

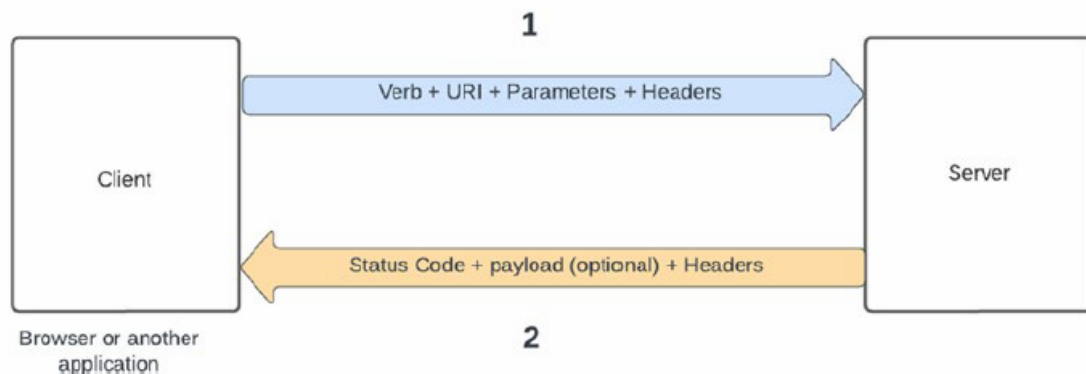


Figure 1-1. A basic HTTP request and its response

HTTP Implementation

HTTP Verbs

RFC 7231 defines the following verbs:

GET: This is the most well-known. It allows you to request a resource from the server and receive a response in the desired format (defined by the headers). The response is **cacheable** (retain information in memory).

HEAD: This verb is similar to the **GET** verb but does not return any payload; a payload is used to request metadata at the requested address. Developers barely use it most of the time, but it's good to know what it is used for. Like GET, the server response is also **cacheable**.

POST: This verb is interesting because it serves multiple purposes. This verb allows the creation of new resources, and its payload is attached to the request's body. Another way to send data is to use the *form-data* technique, which will be described later in this book. The **POST** verb also allows modifying data by adding content to the data (appending data to the resource representation according to the RFC). **The server response is not cacheable unless freshness information** is added to the response headers (max-age or Expires headers). We will discuss it again in the "Request and Response Headers" section.

Chapter 1 Introducing HTTP and REST7

PUT: This verb is confusing because the RFC states that it replaces a resource on the server. Very often, developers confuse **PUT** and **POST** (replace a resource vs. append data to a resource). The server response here is **not cacheable**. However, if a resource doesn't exist, **PUT** should behave as **POST** by creating the resource.

DELETE: This verb is used to delete a resource. The server response is **not cacheable**.

CONNECT: The verb establishes a tunnel to the server through a proxy (a server to which the HTTP request will be delegated and access the server for the requested request). This verb is used for secure requests with *Transport Layer Security (TLS)*, in other words, **HTTPS**. I will also come back to **HTTPS** later in this chapter. I never had to use this verb, and I won't talk about it in this book. The server response is **not cacheable**.

OPTIONS: This verb can be helpful when you want to know what verbs are supported for a given URI. It's also used in the context of *Cross-Origin Resource Sharing (CORS)*, which has its dedicated section further in this book. In the API world, you don't necessarily need to use this verb because you will usually know the available URI for a given endpoint through the *OpenAPI* specification. This will be discussed in the "Extend Your Talent on the Web with REST Architecture Style" section of this chapter. We will also see it together in Chapter 4 when I bring up the API documentation topic. The server response is **not cacheable**.

TRACE: A TRACE request sends a request to the server with no particular payload. This lets you see if intermediate servers, such as proxies, have altered the original request. In the context of APIs, this verb is not used. The server response is **not cacheable**.

RFC 7231 does not describe all the existing verbs, and there are others! **RFC 5789** defines the **PATCH** verb. This RFC can be found here: www.rfc-editor.org/rfc/rfc5789.html.

The **PATCH** verb can be confused with **PUT** and **POST** verbs because they all allow modifying a resource on a server. **PATCH** partially updates a resource (like **POST**) when **PUT** tends to replace a resource.

I see many developers confusing each other. Now you are aware of what the RFCs indicate about these verbs, but see that it is commonly accepted to use **POST** for resource creation or to replace **GET** verb when there are too many parameters in the URI to put them in the body of a **POST**. It's also commonly accepted to use **PUT** to entirely or partially replace a resource even if **PATCH** is made for that. Personally, I rarely use **PATCH**, only when I want to update a single property of a resource (e.g., a date). From the moment I start modifying and altering several properties of a resource (a date, a status, a description, etc.), I instead implement **PUT**.

If you recall, I briefly mentioned HTTP status codes in this section. The following section will discuss how status codes link to HTTP verbs. Some verbs are used essentially with certain HTTP statuses. In the next section, I will list the HTTP statuses and what verbs they can be associated with.

HTTP Status Codes

HTTP status codes are essential in an HTTP request/response between a server and a client. They allow the client, when the server's response has been received, to be informed of the result of the processing by the server. HTTP status codes are also governed by **RFC 7231**. I will not exhaustively

Chapter 1 Introducing HTTP and REST9

detail each HTTP status class and each HTTP code because **RFC 7231** does a pretty good job of doing so, and I won't use all of them in this book. Regarding APIs, status codes are essential for clients to understand what the server is trying to tell us. They provide us with insights on what to do next.

An HTTP status code has three digits. The first digit defines the status category, and there are five categories of HTTP status codes:

- 1xx**: They are purely informational.

- 2xx**: They express that the server received and processed the request successfully.

- 3xx**: They inform the client that the server has proceeded to a redirection, that is, the resource is not at the address indicated, but that the request will be redirected there automatically.

- 4xx**: They mean the request (client-side) is malformed and/or the client (the end user) probably made an input error in their request. 4xx are errors that the client can fix.

- 5xx**: They tell the client that the request on the server has not been completed due to an error.

RFC 7231 is not the only RFC that describes HTTP status codes. However, it describes the codes most often used. **RFC 4918** and **RFC 6585** complete the list, with other codes covering other scenarios.

Table 1-1, taken from **the following RFCs**

RFC 7231: www.rfc-editor.org/rfc/rfc7231

RFC 4918: www.rfc-editor.org/rfc/rfc4918

RFC 6585: www.rfc-editor.org/rfc/rfc6585

Chapter 1 Introducing HTTP and REST10

lists the association between HTTP status codes and HTTP verbs commonly used as industry standards. I won't use all of them in this book; you will not need to know them by heart. On the other hand, knowing their existence is valuable since you will know their existence and how to use them when required. Later in this book, I'll dig deeper into why I'm using some of them in the code samples I provide.